Київський національний університет ім. Тараса Шевченка

Кафедра мережевих та інтернет технологій

**Лабораторна робота № 1**

**Дисципліна: Системне програмування**

**Тема: Потоки. Клас thread. Пріоритети потоків**

Виконав: Студент групи МІТ-41

Полюхович Андрій

**Завдання:**

1. Реалізуйте програму на C#, яка створює два потоки: Перший потік виводить числа від 1 до 40 з паузою 200 мс. Другий потік виводить літери від A до Z з паузою 300 мс. Використайте методи Start(), Sleep() та властивість Name.

Виконання:
Printer.cs

```
using System;
using System.Threading;

namespace Lab1_Task1
{
    // Статичний клас із завданнями для наших потоків
    public static class Printer
    {
        // Завдання 1: вивести числа від 1 до 40
        public static void PrintNumbers()
        {
            for (int i = 1; i <= 40; i++)
            {
                // Виводимо ім'я потоку та число
                Console.WriteLine($"{Thread.CurrentThread.Name}: {i}");
                // Пауза 200 мілісекунд
                Thread.Sleep(200);
            }
            // Повідомлення про завершення
            Console.WriteLine($"\n{Thread.CurrentThread.Name} finished
execution.");
        }

        // Завдання 2: вивести літери англійської абетки
        public static void PrintLetters()
        {
            for (char c = 'A'; c <= 'Z'; c++)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name}: {c}");
                // Пауза 300 мілісекунд
                Thread.Sleep(300);
            }
            Console.WriteLine($"\n{Thread.CurrentThread.Name} finished
execution.");
        }
    }
```

```
}
```

Program.cs:

```
using System;
using System.Threading;

namespace Lab1_Task1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main thread started...\n");

            // Створюємо два потоки, одразу задаємо їм імена
            Thread thread1 = new Thread(Printer.PrintNumbers) { Name =
"Number Thread" };
            Thread thread2 = new Thread(Printer.PrintLetters) { Name =
"Letter Thread" };

            // Запускаємо виконання потоків
            thread1.Start();
            thread2.Start();

            // Змушуємо головний потік чекати завершення обох дочірніх
            thread1.Join();
            thread2.Join();

            Console.WriteLine("\nMain thread finished execution.");
            Console.ReadLine();
        }
    }
}
```

Результат:

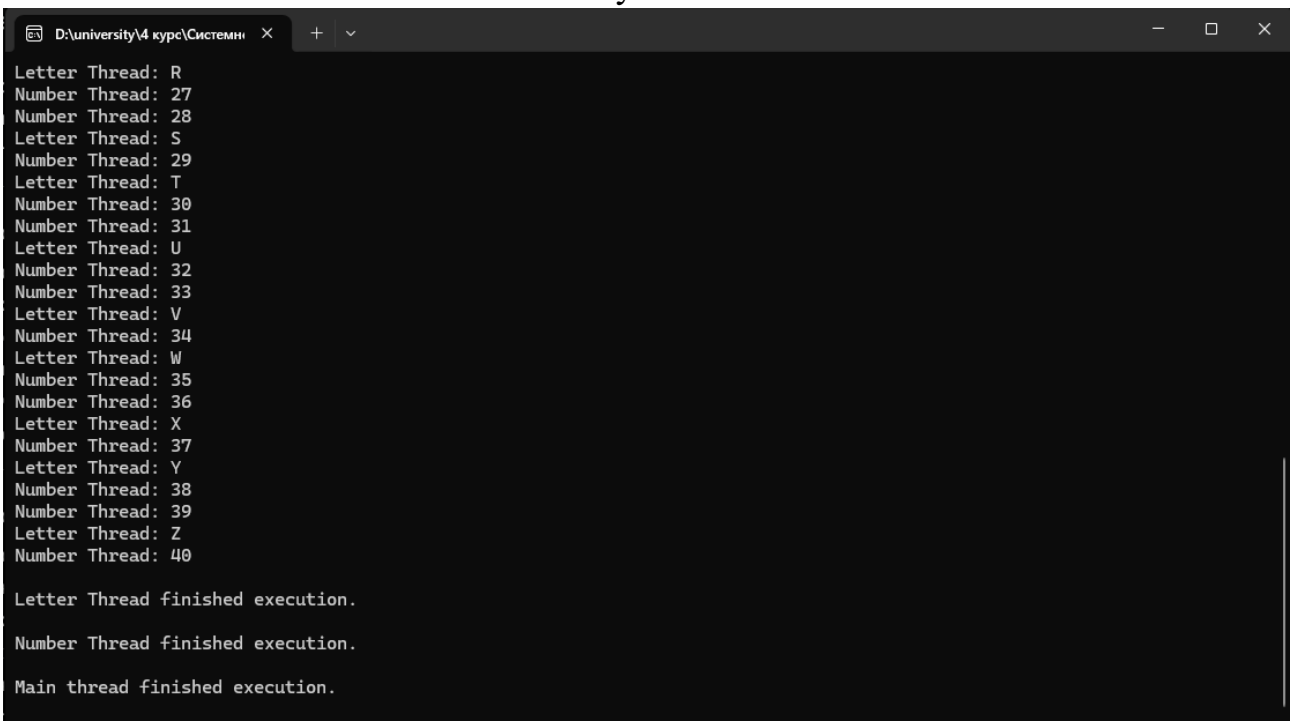

Рисунок 1.1 – Результат виконання завдання 1

Це програма показує базовий принцип багатозадачності. Ми створюємо два потоки, які працюють одночасно: один виводить числа, інший — літери.

Як це працює: Завдяки методу Thread.Sleep(), коли один потік "засинає" на частку секунди, процесор не простоює, а миттєво перемикається на інший потік. Метод Join() потрібен для того, щоб головна програма дочекалася їхнього завершення і не закрила консоль завчасно.

**Питання для звіту**: поясніть, як працює багатозадачність у межах одного

процесу.

У межах однієї запущеної програми ми можемо створити кілька паралельних гілок виконання (потоків), які розв'язують різні задачі. Процесор дуже швидко перемикається між ними. Якщо якась частина програми змушена простоювати (чекає на дані або таймер), операційна система віддає процесорний час іншому потоку цієї ж програми, що робить її роботу набагато ефективнішою.

Завдання:

2. Створіть програму, яка запускає три потоки: Два пріоритетні (звичайні). Один фоновий. Організуйте так, щоб фоновий потік виконував нескінченний цикл з повідомленням у консоль.

Виконання:

Worker.cs

```
using System;
using System.Threading;

namespace Lab1_Task2
{
    public static class Worker
    {
        // Метод для пріоритетних (звичайних) потоків
        public static void NormalThreadTask()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} is
executing (step {i})");
                Thread.Sleep(500); // Пауза півсекунди
            }
            Console.WriteLine($"--- {Thread.CurrentThread.Name} FINISHED -
--");
        }

        // Метод для фонового потоку (нескінченний цикл)
        public static void BackgroundThreadTask()
        {
            int count = 1;
```

```
            while (true)
            {
                Console.WriteLine($"[BG] Background thread is working...
({count++})");
                Thread.Sleep(300);
            }
        }
    }
}
```

Program.cs

```
using System;
using System.Threading;

namespace Lab1_Task2
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main thread started...\n");

            // Створюємо звичайні (пріоритетні) потоки
            Thread normalThread1 = new Thread(Worker.NormalThreadTask) {
Name = "Normal Thread 1" };
            Thread normalThread2 = new Thread(Worker.NormalThreadTask) {
Name = "Normal Thread 2" };

            // Створюємо фоновий потік (обов'язково вказуємо IsBackground
= true)
            Thread bgThread = new Thread(Worker.BackgroundThreadTask) {
Name = "Background Thread", IsBackground = true };

            // Спочатку запускаємо фоновий потік, потім звичайні
            bgThread.Start();
            normalThread1.Start();
            normalThread2.Start();

            // Головний потік чекає ТІЛЬКИ на завершення звичайних потоків
            normalThread1.Join();
            normalThread2.Join();

            // Коли звичайні потоки закінчують роботу, програма доходить
сюди.
            // Після цього фоновий потік буде примусово завершений ОС.
            Console.WriteLine("\nMain thread finished execution.");
        }
    }
}
```

Результат:

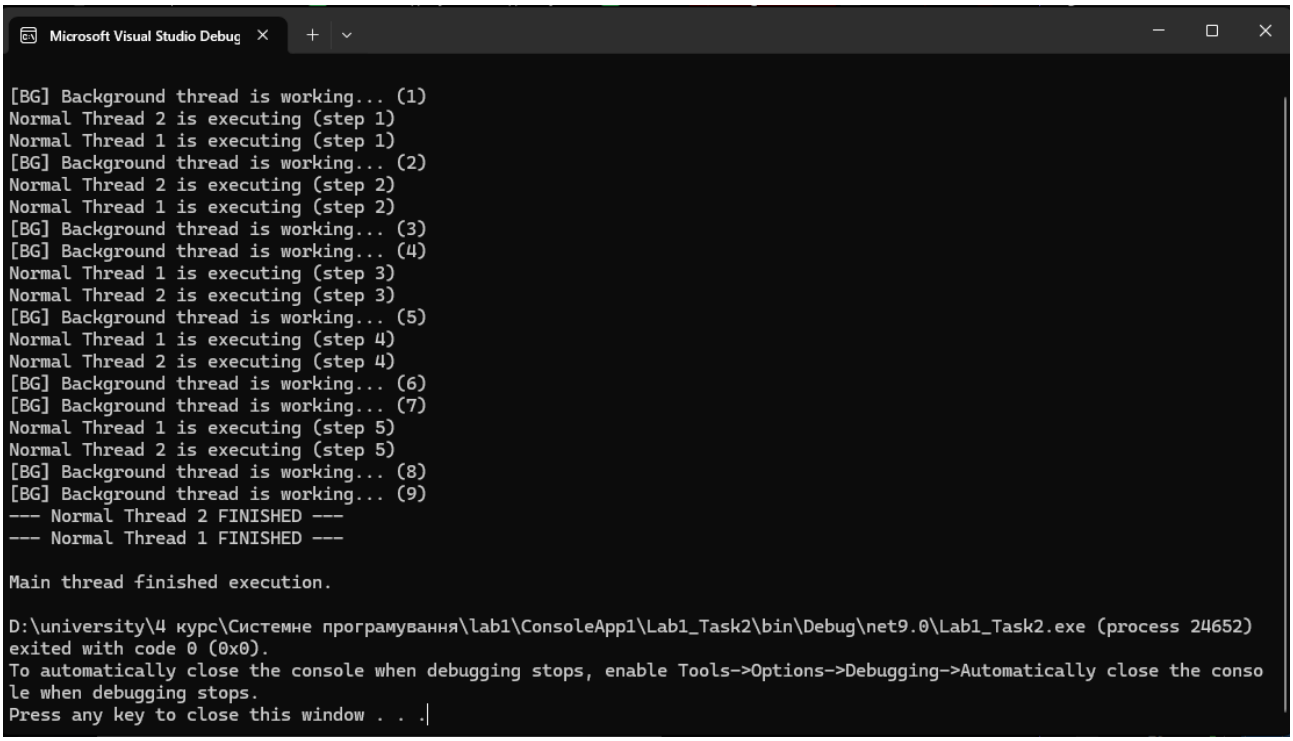


Рисунок 1.2 – Результат виконання завдання 2

Ця програма демонструє різницю між звичайними (пріоритетними) та фоновими потоками.

Як це працює: Звичайні потоки виконують свої 5 кроків і тримають програму відкритою. Фоновий потік працює у нескінченному циклі while(true). Але як тільки звичайні потоки закінчують свою роботу, життєвий цикл програми завершується, і операційна система автоматично "вбиває" фоновий потік, обриваючи його роботу.

**Питання для звіту:** що відбудеться з фоновим потоком після завершення пріоритетних потоків?

Фоновий потік не підтримує "життя" програми. Програма працює лише доти, доки працює хоча б один пріоритетний потік. Щойно останній пріоритетний потік фінішує, фоновий потік автоматично переривається і знищується системою разом із завершенням усього процесу.

Завдання:

3. Створити багатопотокову програму, задати різні пріоритети для потоків. Кожен потік виконує підрахунок до 100 млн. Після завершення роботи програми визначте кількість ітерацій, виконаних кожним потоком, та розрахуйте відсотковий розподіл процесорного часу.

Варіант 4) Потоків – 6 шт. (пріоритети: середній, нижче середнього, вище середнього, найвищий, найнижчий, середній).

Виконання:

MyThread.cs

```
using System.Threading;

namespace Lab1_Task3
{
    // Клас-обгортка для зручного керування потоком
    public class MyThread
    {
        // Лічильник ітерацій конкретного потоку
        public int Count { get; private set; }
        public Thread Thrd { get; private set; }

        // Спільний прапорець для всіх потоків. Якщо хтось дорахував —
змінює на true.
        private static bool stop = false;

        // Конструктор, що приймає ім'я та рівень пріоритету
        public MyThread(string name, ThreadPriority priority)
        {
            Count = 0;
            Thrd = new Thread(Run)
            {
                Name = name,
                Priority = priority
            };
        }

        private void Run()
        {
            // Рахуємо, поки спільний прапорець false і лічильник < 100
мільйонів
            do
            {
                Count++;
            } while (!stop && Count < 100000000);

            // Перший потік, що вийде з циклу, встановить stop = true і
зупинить інших
            stop = true;
        }
    }
}
```

Program.cs

```
using System;
using System.Collections.Generic;
using System.Threading;

namespace Lab1_Task3
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Starting thread race...\n");
```

```
            // Створюємо 6 потоків із різними пріоритетами згідно з 4-м
варіантом
            List<MyThread> threads = new List<MyThread>
            {
                new MyThread("Thread 1 (Normal)", ThreadPriority.Normal),
                new MyThread("Thread 2 (BelowNormal)",
ThreadPriority.BelowNormal),
                new MyThread("Thread 3 (AboveNormal)",
ThreadPriority.AboveNormal),
                new MyThread("Thread 4 (Highest)",
ThreadPriority.Highest),
                new MyThread("Thread 5 (Lowest)", ThreadPriority.Lowest),
                new MyThread("Thread 6 (Normal)", ThreadPriority.Normal)
            };

            // Запуск та очікування всіх потоків
            foreach (var t in threads) t.Thrd.Start();
            foreach (var t in threads) t.Thrd.Join();

            // Підраховуємо загальну суму ітерацій для визначення 100%
            long totalCount = 0;
            foreach (var t in threads) totalCount += t.Count;

            Console.WriteLine("--- RESULTS ---");

            // Виводимо статистику (ім'я, ітерації та відсоток
процесорного часу)
            foreach (var t in threads)
            {
                double percentage = (double)t.Count / totalCount * 100;
                Console.WriteLine($"{t.Thrd.Name,-25} | Iterations:
{t.Count,-10} | CPU Time: {percentage:F2}%");
            }

            Console.WriteLine("\nMain thread finished execution.");
            Console.ReadLine();
        }
    }
}
```

Результат:

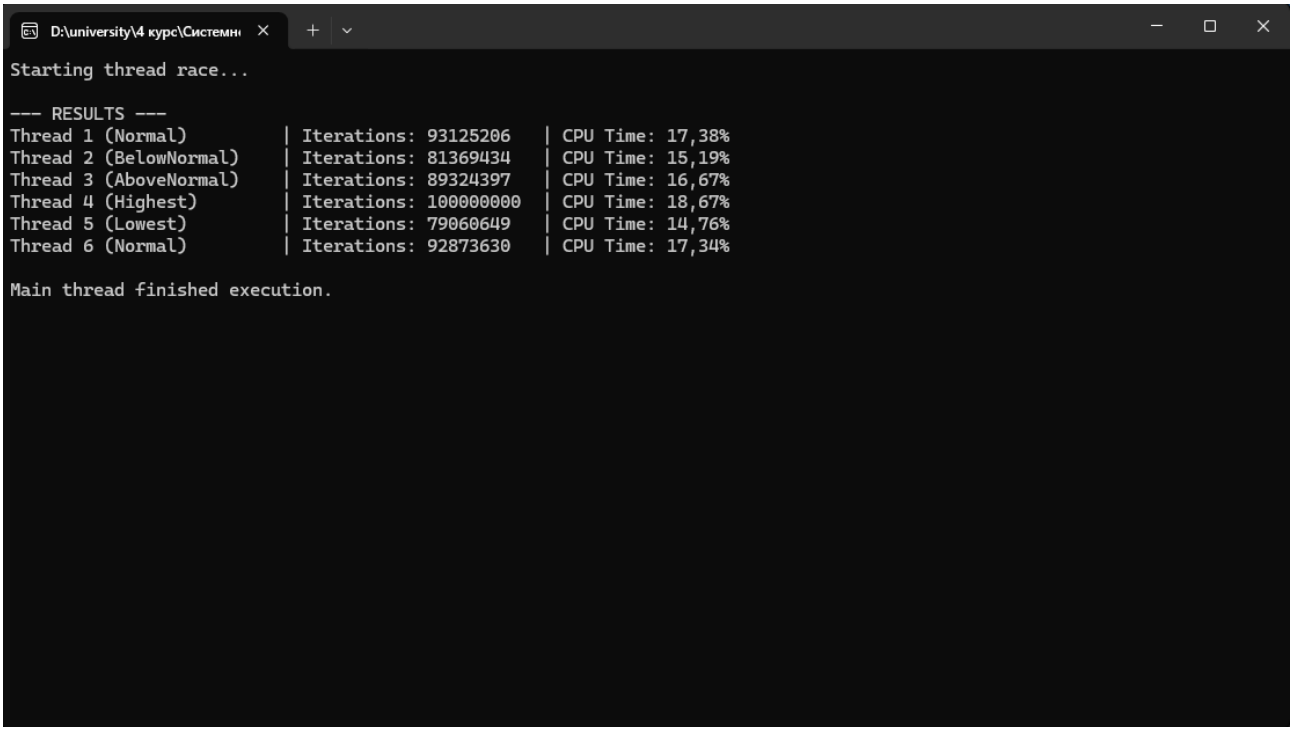


Рисунок 1.3 – Результат виконання завдання 3

Ця програма влаштовує "перегони" між 6 потоками з різними пріоритетами.

Як це працює: Всі потоки одночасно починають рахувати. Як тільки найшвидший з них доходить до 100 мільйонів, він перемикає спільну змінну stop на true. Інші потоки бачать це і теж миттєво зупиняються. Після цього ми порівнюємо, скільки ітерацій встиг зробити кожен потік, і бачимо, що потоки з вищим пріоритетом зазвичай встигають більше, бо ОС виділяє їм більше часу.

**Питання для звіту:** чи завжди високопріоритетний потік отримує найбільше часу ЦП? Які фактори можуть впливати на планування?

Високий пріоритет дає потоку перевагу, але не гарантує 100% часу. На розподіл впливає те, чим займається потік: якщо він зупиняється або чекає на якийсь ресурс, замість нього відразу починає виконуватися фоновий або низькопріоритетний потік. Також велику роль відіграють внутрішні алгоритми планування самої Windows (чи іншої ОС), які стежать за загальним навантаженням на систему.

Завдання:

4. Високий рівень: Розробіть консольний застосунок, який:
   - Дозволяє користувачу ввести кількість потоків та їхні пріоритети.
   - Запускає кожен потік із власним завданням (наприклад, підрахунок до заданого числа).
   - Виводить у реальному часі хід виконання кожного потоку (лічильник, стан).
   - Після завершення – формує таблицю розподілу процесорного часу між потоками у %.

Виконання:
ThreadWorker.cs

```
using System.Threading;

namespace Lab1_Task4
{
    public class ThreadWorker
    {
        public int Id { get; }
        public ThreadPriority Priority { get; }
        public long Count { get; private set; }
        public bool IsFinished { get; private set; }

        // Нова властивість, щоб знати, чи цей потік виграв гонку
        public bool IsWinner { get; private set; }
        public Thread Thrd { get; }

        private readonly long _target;

        // Спільний прапорець для всіх потоків (volatile гарантує миттєве
оновлення для всіх ядер ЦП)
        public static volatile bool StopRace = false;

        public ThreadWorker(int id, ThreadPriority priority, long target)
        {
            Id = id;
            Priority = priority;
            _target = target;
            Count = 0;
            IsFinished = false;
            IsWinner = false;

            Thrd = new Thread(Run) { Priority = priority };
        }

        private void Run()
        {
            // Потік працює, поки не досягне мети АБО поки хтось інший не
виграє гонку
            while (Count < _target && !StopRace)
            {
                Count++;
            }

            // Якщо цей потік вийшов з циклу, бо САМ досяг мети першим
            if (Count >= _target && !StopRace)
            {
                StopRace = true; // Зупиняємо всіх інших
                IsWinner = true; // Оголошуємо себе переможцем
            }

            IsFinished = true; // Ставимо статус завершення
        }
    }
}
```

Program.cs

```
using System;
using System.Collections.Generic;
using System.Threading;

namespace Lab1_Task4
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("--- THREAD CONFIGURATION ---");
            Console.Write("Enter the target number for counting (e.g., 
100000000): ");
            long target = long.Parse(Console.ReadLine());

            Console.Write("Enter the number of threads: ");
            int numThreads = int.Parse(Console.ReadLine());

            List<ThreadWorker> workers = new List<ThreadWorker>();

            for (int i = 1; i <= numThreads; i++)
            {
                Console.WriteLine($"\nSelect priority for Thread {i}:");
                Console.WriteLine("1 - Lowest\n2 - BelowNormal\n3 - 
Normal\n4 - AboveNormal\n5 - Highest");
                Console.Write("Your choice (1-5): ");
                int choice = int.Parse(Console.ReadLine());

                ThreadPriority priority = choice switch
                {
                    1 => ThreadPriority.Lowest,
                    2 => ThreadPriority.BelowNormal,
                    3 => ThreadPriority.Normal,
                    4 => ThreadPriority.AboveNormal,
                    5 => ThreadPriority.Highest,
                    _ => ThreadPriority.Normal
                };

                workers.Add(new ThreadWorker(i, priority, target));
            }

            Console.Clear();
            Console.WriteLine("Starting execution...\n");

            // Скидаємо прапорець гонки перед стартом
            ThreadWorker.StopRace = false;

            int startRow = Console.CursorTop;

            foreach (var w in workers) w.Thrd.Start();

            bool allDone = false;
            while (!allDone)
            {
                allDone = true;
                Console.SetCursorPosition(0, startRow);

                foreach (var w in workers)
                {
```

```
                    // Визначаємо статус для красивого виводу
                    string status;
                    if (w.IsWinner) status = "[WINNER]  ";
                    else if (w.IsFinished) status = "[STOPPED] ";
                    else status = "[RUNNING] ";

                    double progress = (double)w.Count / target * 100;

                    Console.WriteLine($"Thread {w.Id} ({w.Priority,-11}) |
Status: {status} | Counter: {w.Count,-12} | Progress: {progress,6:F2}%");

                    if (!w.IsFinished) allDone = false;
                }

                Thread.Sleep(50);
            }

            foreach (var w in workers) w.Thrd.Join();

            // Виведення фінальної таблиці з переможцем
            Console.WriteLine("\n--- RACE RESULTS & CPU TIME DISTRIBUTION
---");
            long totalCount = 0;
            foreach (var w in workers) totalCount += w.Count;

            // Сортуємо список, щоб переможець завжди відображався першим
у таблиці результатів (за бажанням)
            // workers.Sort((a, b) => b.Count.CompareTo(a.Count));

            foreach (var w in workers)
            {
                double timePercentage = (double)w.Count / totalCount *
100;

                // Додаємо медаль для переможця
                string place = w.IsWinner ? "[1st PLACE] " : "[STOPPED]
";

                Console.WriteLine($"{place} Thread {w.Id} ({w.Priority,-
11}) received ~{timePercentage:F2}% of CPU time.");
            }

            Console.WriteLine("\nPress any key to exit.");
            Console.ReadKey();
        }
    }
}
```

Результат:

```
D:\university\4 курс\Системн

--- THREAD CONFIGURATION ---
Enter the target number for counting (e.g., 100000000): 100000000
Enter the number of threads: 4

Select priority for Thread 1:
1 - Lowest
2 - BelowNormal
3 - Normal
4 - AboveNormal
5 - Highest
Your choice (1-5): 1

Select priority for Thread 2:
1 - Lowest
2 - BelowNormal
3 - Normal
4 - AboveNormal
5 - Highest
Your choice (1-5): 5

Select priority for Thread 3:
1 - Lowest
2 - BelowNormal
3 - Normal
4 - AboveNormal
5 - Highest
Your choice (1-5): 3

Select priority for Thread 4:
1 - Lowest
2 - BelowNormal
3 - Normal
4 - AboveNormal
5 - Highest
Your choice (1-5): 4
```

Рисунок 1.4 – користувач вводить кількість потоків та їхні пріоритети

```
D:\university\4 курс\Системн

Starting execution...

Thread 1 (Lowest     ) | Status: [STOPPED]  | Counter: 87964244     | Progress:  87,96%
Thread 2 (Highest    ) | Status: [WINNER]   | Counter: 100000000    | Progress: 100,00%
Thread 3 (Normal     ) | Status: [STOPPED]  | Counter: 96241630     | Progress:  96,24%
Thread 4 (AboveNormal) | Status: [STOPPED]  | Counter: 93647596     | Progress:  93,65%

--- RACE RESULTS & CPU TIME DISTRIBUTION ---
[STOPPED]    Thread 1 (Lowest     ) received ~23,28% of CPU time.
[1st PLACE]  Thread 2 (Highest    ) received ~26,47% of CPU time.
[STOPPED]    Thread 3 (Normal     ) received ~25,47% of CPU time.
[STOPPED]    Thread 4 (AboveNormal) received ~24,78% of CPU time.

Press any key to exit.
```

Рисунок 1.5 – Результат виконання завдання 4

Ця програма дозволяє гнучко налаштувати потоки та в реальному часі спостерігати за їхньою роботою.

Як це працює: Уся логіка "гонки" базується на єдиній спільній для всіх потоків змінній-прапорці StopRace (з модифікатором volatile для миттєвого оновлення в

пам'яті). Під час роботи кожен потік на кожному кроці циклу перевіряє не лише свій лічильник, а й те, чи не натиснув хтось інший "червону кнопку" зупинки. Як тільки найшвидший потік (завдяки вищому пріоритету) першим доходить до фінішу, він миттєво змінює значення StopRace на true і отримує статус переможця. У цей же момент усі інші потоки бачать зміну прапорця, їхній цикл примусово переривається, і вони назавжди зупиняються на тих відсотках процесорного часу, до яких встигли дорахувати.

Висновок:
Під час виконання лабораторної роботи закріплено навички роботи з багатопотоковістю в C# за допомогою класу Thread. Ми успішно реалізували створення та синхронізацію потоків методами Start(), Sleep() та Join(). Практично доведено, що фонові потоки автоматично припиняють роботу після завершення пріоритетних, а зміна пріоритету потоку впливає на його частку процесорного часу, хоча остаточний розподіл ресурсів завжди контролюється операційною системою.